แผนโครงการพัฒนา: ผู้ช่วยสอน AI สำหรับการเขียนโปรแกรมเชิงวัตถุ (AI Tutor for Object-Oriented Programming)

บทที่ 1: ภาพรวมโครงการ (Project Overview)

1.1 หลักการและเหตุผล: จากความท้าทายสู่โอกาสด้วย AI (Rationale: From Challenge to Opportunity with AI)

การเรียนการสอนการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming - OOP) ในระดับมหาวิทยาลัยนั้นเต็มไปด้วยความท้าทายที่เฉพาะเจาะจง นักศึกษาจำนวนมากประสบปัญหาในการทำความเข้าใจแนวคิดเชิงนามธรรม (Abstract Concepts) เช่น การห่อหุ้ม (Encapsulation), การสืบทอด (Inheritance), และการพ้องรูป (Polymorphism). ความยากลำบากนี้เกิดจากการที่ OOP เป็นกระบวนทัศน์ (Paradigm) ที่แตกต่างอย่างสิ้นเชิงจากการเขียนโปรแกรมเชิงกระบวนคำสั่ง (Procedural Programming) ที่นักศึกษาส่วนใหญ่คุ้นเคย ซึ่งต้องอาศัยการปรับเปลี่ยนวิธีคิดหรือ "Paradigm Shift" ที่อาจใช้เวลานาน. ปัญหาไม่ได้จำกัดอยู่แค่ความเข้าใจใน **синтаксис** แต่หยั่งลึกลงไปถึงการสร้างแบบจำลองความคิด (Mental Model) ที่ไม่ถูกต้อง ซึ่งส่งผลโดยตรงต่อคุณภาพของการออกแบบซอฟต์แวร์ในระยะยาว. นอกจากนี้ ปัจจัยอื่นๆ เช่น ความซับซ้อนของเครื่องมือที่ใช้ในอุตสาหกรรม และระดับความวิตกกังวลที่สูงของผู้เรียน ก็เป็นอุปสรรคสำคัญที่ทำให้การเรียนรู้ OOP ไม่ราบรื่นเท่าที่ควร.

เพื่อตอบสนองต่อความท้าทายเหล่านี้ โครงการนี้จึงนำเสนอการพัฒนา "ผู้ช่วยสอน AI" (AI Tutor) ซึ่งเป็นเครื่องมือที่สามารถปฏิวัติกระบวนการเรียนรู้ OOP ได้อย่างมีนัยสำคัญ. AI Tutor สามารถมอบประสบการณ์การเรียนรู้ที่ปรับให้เข้ากับแต่ละบุคคล (Personalized Learning) โดยระบบจะวิเคราะห์รูปแบบการเรียนรู้และระบุจุดอ่อนของนักศึกษาแต่ละคน เพื่อนำเสนอเนื้อหา, คำอธิบาย, และแบบฝึกหัดที่เหมาะสมกับระดับความเข้าใจของพวกเขาโดยเฉพาะ. คุณสมบัติที่สำคัญที่สุดคือความสามารถในการให้คำติชมแบบทันที (Immediate Feedback) และพร้อมให้บริการตลอด 24 ชั่วโมง. สภาพแวดล้อมการเรียนรู้เช่นนี้เปรียบเสมือน "พื้นที่ปลอดภัยสำหรับการล้มเหลว" (Safe Space for Failure) ที่นักศึกษาสามารถทดลองแนวคิดเชิงนามธรรมที่ซับซ้อน, ทำผิดพลาด, และเรียนรู้จากข้อผิดพลาดเหล่านั้นได้ทันที โดยปราศจากแรงกดดันทางจิตใจหรือความกังวลที่จะถูกตัดสินจากเพื่อนร่วมชั้นหรืออาจารย์. การมีปฏิสัมพันธ์ที่ต่อเนื่องและเป็นส่วนตัวนี้ไม่เพียงแต่ช่วยแก้ไขความเข้าใจผิดได้อย่างรวดเร็ว แต่ยังช่วยลดความวิตกกังวลและสร้างความมั่นใจ ซึ่งเป็นปัจจัยสำคัญที่เอื้อต่อการสร้างแบบจำลองความคิดที่ถูกต้องและส่งเสริมความเข้าใจในหลักการ OOP อย่างแท้จริง.

1.2 วิสัยทัศน์และเป้าหมายของผลิตภัณฑ์ (Product Vision and Goals)

วิสัยทัศน์ (Vision): "สร้างผู้ช่วยสอน AI อัจฉริยะที่ทำให้นักศึกษาทุกคนสามารถเข้าถึงการเรียนรู้ OOP คุณภาพสูงได้ทุกที่ทุกเวลา ส่งเสริมความเข้าใจในหลักการออกแบบเชิงวัตถุอย่างลึกซึ้ง และสร้างความมั่นใจในการเขียนโค้ดภาษา Python"

เป้าหมายหลัก (Primary Goals):

- **Goal 1: Enhance Conceptual Understanding:** พัฒนาเครื่องมือที่สามารถอธิบายแนวคิด OOP ที่ซับซ้อนด้วยวิธีการที่หลากหลายและปรับให้เข้ากับผู้เรียนแต่ละคนได้ เช่น การใช้คำอุปมาอุปไมย, ตัวอย่างโค้ดที่ชัดเจน, และการให้คำแนะนำแบบทีละขั้นตอน เพื่อให้นักศึกษาสามารถสร้างความเข้าใจที่ถูกต้องและยั่งยืน.

- **Goal 2: Promote High-Quality Coding Practices:** ยกระดับการสอนจากการมุ่งเน้นเพียงแค่ "โค้ดที่ทำงานได้" ไปสู่ "โค้ดที่มีคุณภาพสูง" โดย AI Tutor จะให้คำแนะนำเกี่ยวกับหลักการออกแบบที่ดี เช่น Coupling, Cohesion, และหลักการ SOLID ตั้งแต่ระยะเริ่มต้นของการเรียนรู้ เพื่อปลูกฝังแนวปฏิบัติที่ดีที่สุดให้กับนักศึกษา.
- **Goal 3: Increase Student Engagement:** สร้างประสบการณ์การเรียนรู้เชิงโต้ตอบ (Interactive Learning) ผ่านแบบฝึกหัดที่ท้าทาย, การให้คำใบ้แบบปรับเปลี่ยนได้, และองค์ประกอบของเกม (Gamification) เพื่อกระตุ้นและรักษาความสนใจของผู้เรียน ทำให้การเรียนรู้เป็นเรื่องสนุกและไม่น่าเบื่อ.

เป้าหมายรอง (Secondary Goals):

- ลดภาระงานของอาจารย์ผู้สอนในการตรวจการบ้านเบื้องต้นและตอบคำถามที่พบบ่อย เพื่อให้อาจารย์สามารถทุ่มเทเวลาไปกับการให้คำปรึกษาเชิงลึกแก่นักศึกษาเป็นรายบุคคลได้มากขึ้น.
- ทำหน้าที่เป็นเครื่องมือเสริมการเรียนรู้นอกห้องเรียนที่เชื่อถือได้ ซึ่งนักศึกษาสามารถเข้าถึงเพื่อทบทวนบทเรียน, ฝึกฝน, หรือขอความช่วยเหลือได้ทุกเมื่อที่ต้องการ.

1.3 กลุ่มเป้าหมายและผลลัพธ์ที่คาดหวัง (Target Audience and Expected Outcomes)

กลุ่มเป้าหมายหลัก (Target Audience): นักศึกษาระดับปริญญาตรี ชั้นปีที่ 1 หรือ 2 ในสาขาวิทยาการคอมพิวเตอร์, วิศวกรรมซอฟต์แวร์, เทคโนโลยีสารสนเทศ หรือสาขาที่เกี่ยวข้อง ที่กำลังลงทะเบียนเรียนวิชาการเขียนโปรแกรมเชิงวัตถุด้วยภาษา Python เป็นครั้งแรก.

ตัวชี้วัดความสำเร็จ (Key Performance Indicators - KPIs):

- Academic Performance:
  - ลดอัตราการถอนรายวิชาหรือสอบไม่ผ่านในวิชา OOP ลง 15% เมื่อเทียบกับค่าเฉลี่ยในปีก่อนหน้า.
  - เพิ่มคะแนนเฉลี่ยในส่วนของการสอบปฏิบัติ (Practical Exams) และโปรเจกต์ที่เกี่ยวข้องกับการเขียนโค้ดขึ้น 10%.
- User Engagement:
  - ผู้ใช้งานที่ลงทะเบียนแล้ว 70% กลับมาใช้งานระบบอย่างน้อย 3 ครั้งต่อสัปดาห์ตลอดภาคการศึกษา.
  - ค่าเฉลี่ยของระยะเวลาที่ใช้ในระบบต่อครั้ง (Average Session Time) อยู่ที่ 20 นาทีขึ้นไป.
- User Satisfaction:
  - คะแนนความพึงพอใจของผู้ใช้งานที่วัดจากแบบสำรวจหลังการใช้งาน (User Satisfaction Survey) ต้องไม่ต่ำกว่า 4.2 จาก 5.0.

บทที่ 2: ชุดเทคโนโลยีที่ใช้ (Technology Stack)

2.1 สถาปัตยกรรมระบบ (System Architecture)

สถาปัตยกรรมของระบบจะถูกออกแบบตามหลักการ 3-Tier Architecture ซึ่งประกอบด้วยสามส่วนหลักที่ทำงานประสานกันเพื่อมอบประสบการณ์การใช้งานที่ราบรื่นและมีประสิทธิภาพ การไหลของข้อมูลจะเป็นดังนี้:

1. **ส่วนติดต่อผู้ใช้ (Frontend):** พัฒนาด้วยเฟรมเวิร์ก Streamlit ทำหน้าที่เป็นส่วนที่ผู้ใช้มีปฏิสัมพันธ์ด้วยโดยตรง รับข้อมูลนำเข้า (Input) จากผู้ใช้ เช่น คำถามเกี่ยวกับแนวคิด OOP หรือโค้ดที่ต้องการวิเคราะห์ แล้วส่งคำร้องขอ (Request) ไปยังส่วน Backend.

2. **ส่วนตรรกะแอปพลิเคชัน (Backend):** พัฒนาด้วยภาษา Python โดยมี LangChain เป็นแกนหลักในการประสานงาน (Orchestrator). ส่วนนี้จะรับคำร้องขอจาก Frontend, จัดการสร้างและส่ง Prompt ไปยัง LLM API, ประมวลผลการตอบกลับ, และในเฟสที่ 1 จะมีการดึงข้อมูลจาก Vector Database เพื่อประกอบการตอบคำถาม.

3. **ส่วน AI และข้อมูล (AI & Data Layer):** ประกอบด้วยบริการภายนอกคือ Large Language Model (LLM) API เช่น Google Gemini หรือ OpenAI API และฐานข้อมูลเวกเตอร์ (Vector Database) เช่น FAISS (สำหรับช่วงเริ่มต้น) หรือ Pinecone ซึ่งทำหน้าที่จัดเก็บและค้นหาข้อมูลความรู้ที่เกี่ยวข้องกับ OOP.

2.2 Large Language Model (LLM) ที่เลือกใช้

จากการวิเคราะห์และเปรียบเทียบโมเดลภาษาขนาดใหญ่ (LLM) ที่มีอยู่ในปัจจุบัน พบว่าไม่มีโมเดลใดโมเดลหนึ่งที่เหมาะสมที่สุดสำหรับทุกภารกิจ. แต่ละโมเดลมีจุดแข็งและจุดอ่อนที่แตกต่างกันในด้านความเร็วในการประมวลผล, ค่าใช้จ่ายในการใช้งาน, ขนาดของ Context Window, และความสามารถในการให้เหตุผลเชิงลึกที่ซับซ้อน. โครงการนี้มีแผนการพัฒนา 3 เฟส ซึ่งแต่ละเฟสมีความต้องการที่แตกต่างกันอย่างชัดเจน: เฟส 1 (Q&A) ต้องการความเร็วและความแม่นยำในการดึงข้อมูล, เฟส 2 (สร้างแบบฝึกหัด) ต้องการความคิดสร้างสรรค์และการยึดตามโครงสร้าง, ในขณะที่เฟส 3 (วิเคราะห์โค้ด) ต้องการความสามารถในการให้เหตุผลเชิงตรรกะขั้นสูง.

การใช้โมเดลที่ทรงพลังและมีค่าใช้จ่ายสูงเพียงตัวเดียวสำหรับทุกงานจะเป็นการสิ้นเปลืองทรัพยากรโดยไม่จำเป็นสำหรับงานง่ายๆ เช่น การตอบคำถามพื้นฐาน ในทางกลับกัน การใช้โมเดลที่รวดเร็วและราคาถูกเพียงตัวเดียวก็อาจมีประสิทธิภาพไม่เพียงพอสำหรับงานวิเคราะห์โค้ดที่ซับซ้อนในเฟสที่ 3. ดังนั้น สถาปัตยกรรมที่เหมาะสมที่สุดไม่ใช่การเลือก LLM เพียงตัวเดียว แต่เป็นการออกแบบระบบที่สามารถเลือกใช้ LLM หลายตัวได้อย่างชาญฉลาด (Multi-Model Strategy). สิ่งนี้หมายถึงการสร้าง "Routing Layer" ในฝั่ง Backend ซึ่งจะทำหน้าที่วิเคราะห์ประเภทของคำร้องขอจากผู้ใช้และส่งต่อไปยัง LLM ที่เหมาะสมที่สุดสำหรับงานนั้นๆ กลยุทธ์ "Model Routing" นี้จะช่วยเพิ่มประสิทธิภาพทั้งในด้านความเร็ว (ใช้โมเดลเร็วสำหรับงานง่าย) และด้านค่าใช้จ่าย (สงวนโมเดลราคาแพงไว้สำหรับงานซับซ้อน).

ข้อเสนอแนะในการเลือกใช้ LLM สำหรับแต่ละเฟส:

- **สำหรับเฟส 1 และ 2 (Q&A Bot และ Interactive Exercise Generator):** แนะนำให้ใช้ **Google Gemini 2.0 Flash**. เหตุผลคือโมเดลนี้มีความเร็วในการตอบสนองสูง, มีค่าใช้จ่ายต่อโทเค็นที่ต่ำ, และมีประสิทธิภาพเพียงพอสำหรับงานตอบคำถามโดยใช้ RAG และการสร้างเนื้อหาตาม Template ที่กำหนดไว้.

- **สำหรับเฟส 3 (Code Analysis Tool):** แนะนำให้ใช้ **OpenAI GPT-5-Codex** หรือ **Anthropic Claude 3.7 Sonnet**. เหตุผลคือโมเดลเหล่านี้ได้รับการยอมรับว่ามีความสามารถในการให้เหตุผลเชิงลึก (Deep Reasoning) และความเข้าใจในโครงสร้างโค้ดที่ซับซ้อนสูงกว่า ซึ่งเป็นสิ่งจำเป็นอย่างยิ่งสำหรับการตรวจจับ "Code Smells" และให้คำแนะนำเชิงคุณภาพเกี่ยวกับการออกแบบซอฟต์แวร์.

ตารางที่ 2.1: เปรียบเทียบคุณสมบัติของ LLM ที่พิจารณา

Model	Provider	Strengths	Weaknesses	Cost (per M tokens)	Best Fit for Phase
Gemini 2.0 Flash	Google	ความเร็วสูง, ค่าใช้จ่ายต่ำ, Multimodal	ความสามารถในการให้เหตุผลซับซ้อนน้อยกว่า	ต่ำมาก	1, 2
GPT-5 mini	OpenAI	สมดุลดีระหว่างความเร็วและความแม่นยำ	อาจไม่ดีที่สุดสำหรับงานที่ซับซ้อนมาก	ปานกลาง	1, 2
GPT-5-Codex	OpenAI	ความสามารถในการวิเคราะห์โค้ดสูงมาก	ความเร็วต่ำกว่า, ค่าใช้จ่ายสูง	สูง	3
Claude 3.7 Sonnet	Anthropic	ผลลัพธ์มีโครงสร้างชัดเจน, ทำตามคำสั่งได้ดี	ค่าใช้จ่ายสูงกว่า Flash	ปานกลาง-สูง	3

2.3 เฟรมเวิร์กและไลบรารี (Frameworks and Libraries)

- **Backend & Orchestration: LangChain** จะถูกใช้เป็นเฟรมเวิร์กหลักในการพัฒนา Backend. LangChain เป็นมาตรฐานอุตสาหกรรมสำหรับการสร้างแอปพลิเคชันที่ขับเคลื่อนด้วย LLM และมีส่วนประกอบสำเร็จรูปมากมาย เช่น Document Loaders, Text Splitters, และ Retrievers ที่จะช่วยเร่งกระบวนการพัฒนาสถาปัตยกรรม RAG ในเฟส 1 ได้อย่างมาก.

- **Frontend Prototyping: Streamlit** ถูกเลือกใช้สำหรับสร้างส่วนติดต่อผู้ใช้. จุดเด่นของ Streamlit คือความสามารถในการสร้างเว็บแอปพลิเคชันเชิงโต้ตอบได้อย่างรวดเร็วโดยใช้โค้ด Python เพียงอย่างเดียว โดยไม่จำเป็นต้องมีความรู้ด้าน HTML, CSS, หรือ JavaScript ซึ่งทำให้ทีมพัฒนาสามารถสร้างต้นแบบ (Prototype) ที่ใช้งานได้จริงในเวลาอันสั้น.

- **Vector Database: FAISS (Facebook AI Similarity Search)** จะถูกใช้ในช่วงเริ่มต้นของโครงการ. FAISS เป็นไลบรารีที่มีประสิทธิภาพสูงสำหรับการค้นหาความคล้ายคลึงของเวกเตอร์ และสามารถทำงานบนเครื่องคอมพิวเตอร์ local ได้โดยไม่มีค่าใช้จ่ายเพิ่มเติม. หากในอนาคตโครงการต้องการขยายขนาด (Scale) เพื่อรองรับข้อมูลจำนวนมหาศาล ก็สามารถเปลี่ยนไปใช้บริการบนคลาวด์ เช่น **Pinecone** ได้อย่างไม่ยากนัก.

2.4 เครื่องมือและการจัดการโครงการ (Tooling and Project Management)

- **Version Control: Git** และ **GitHub** จะถูกใช้เป็นเครื่องมือหลักในการควบคุมเวอร์ชันของซอร์สโค้ด. นี่เป็นมาตรฐานอุตสาหกรรมที่ช่วยในการติดตามการเปลี่ยนแปลง, ทำงานร่วมกันในทีม, และสำรองข้อมูลโค้ดได้อย่างมีประสิทธิภาพ.

- **Dependency Management: Python Virtual Environments (venv)** จะถูกใช้เพื่อสร้างสภาพแวดล้อมที่แยกจากกันสำหรับโปรเจกต์นี้โดยเฉพาะ พร้อมทั้งใช้ไฟล์ requirements.txt ในการจัดการไลบรารีและเวอร์ชันที่ใช้ทั้งหมด เพื่อป้องกันปัญหาความขัดแย้งของ dependencies และทำให้การติดตั้งโปรเจกต์บนเครื่องอื่นเป็นไปอย่างราบรื่น.

- **Code Quality: Ruff** (หรือ Flake8) จะถูกใช้สำหรับ Linting เพื่อตรวจสอบข้อผิดพลาดและสไตล์ของโค้ด และ **Black** จะถูกใช้สำหรับ Code Formatting เพื่อจัดรูปแบบโค้ดให้เป็นมาตรฐานเดียวกันทั่วทั้งโปรเจกต์ การใช้เครื่องมือเหล่านี้จะช่วยรักษาคุณภาพและความสามารถในการบำรุงรักษาโค้ดในระยะยาว.

บทที่ 3: แผนการพัฒนาแบบแบ่งเฟส (Detailed Phase-by-Phase Plan)

3.1 เฟส 1: MVP Q&A Bot (ระยะเวลา: 4 สัปดาห์)

วัตถุประสงค์: สร้างผลิตภัณฑ์ขั้นต่ำที่ใช้งานได้ (Minimum Viable Product - MVP) ในรูปแบบของ Chatbot ที่สามารถตอบคำถามพื้นฐานเกี่ยวกับแนวคิด OOP ในภาษา Python ได้อย่างถูกต้องและรวดเร็ว โดยอาศัยคลังความรู้ (Knowledge Base) ที่ทีมงานได้รวบรวมและจัดเตรียมไว้ล่วงหน้า.

สถาปัตยกรรมหลัก: Retrieval-Augmented Generation (RAG).

แผนการดำเนินงานรายสัปดาห์:

- **สัปดาห์ที่ 1: Project Setup & Data Curation**
    - **กิจกรรม:** ตั้งค่า Git repository บน GitHub, สร้างสภาพแวดล้อมเสมือน (virtual environment), และติดตั้งไลบรารีที่จำเป็นในเบื้องต้น.
    - **กิจกรรม:** รวบรวม, คัดเลือก, และจัดระเบียบแหล่งข้อมูลคุณภาพสูงเกี่ยวกับ OOP ใน Python เช่น บทจากตำราเรียน, เอกสารประกอบการสอนของมหาวิทยาลัย, และบทความทางเทคนิคที่เชื่อถือได้.

- **สัปดาห์ที่ 2: Indexing Pipeline Development**

    - **กิจกรรม:** พัฒนาสคริปต์สำหรับกระบวนการ Indexing โดยใช้ LangChain Document Loaders เพื่อนำเข้าข้อมูลจากแหล่งต่างๆ.
    - **กิจกรรม:** ใช้ RecursiveCharacterTextSplitter เพื่อแบ่งเอกสารขนาดใหญ่ให้เป็นส่วนย่อย (Chunks) ที่มีขนาดเหมาะสมต่อการทำ Embedding และการค้นหา.
    - **กิจกรรม:** สร้าง Embeddings จากแต่ละ Chunk และจัดเก็บลงใน FAISS Vector Store ที่ทำงานบนเครื่อง local.
- **สัปดาห์ที่ 3: Retrieval & Generation Chain Development**
    - **กิจกรรม:** สร้าง Retriever ใน LangChain เพื่อดึงข้อมูล Chunks ที่เกี่ยวข้องกับคำถามของผู้ใช้จาก Vector Store.
    - **กิจกรรม:** ออกแบบ Prompt Template ที่มีประสิทธิภาพสำหรับ RAG chain โดยระบุบทบาทของ AI และวิธีการใช้ข้อมูลที่ดึงมา (Context).
    - **กิจกรรม:** เชื่อมต่อระบบกับ Gemini 2.0 Flash API และสร้าง Chain การตอบคำถามที่สมบูรณ์.
- **สัปดาห์ที่ 4: UI Development & Initial Testing**
    - **กิจกรรม:** สร้างส่วนติดต่อผู้ใช้แบบแชทพื้นฐานด้วย Streamlit โดยใช้คอมโพเนนต์ st.chat_input และ st.chat_message.
    - **กิจกรรม:** เชื่อมต่อ UI ที่สร้างขึ้นกับ Backend RAG chain เพื่อให้สามารถรับคำถามและแสดงคำตอบได้.
    - **กิจกรรม:** ทำการทดสอบระบบแบบ End-to-End เพื่อตรวจสอบการทำงานทั้งหมดและแก้ไขข้อบกพร่องเบื้องต้น.

3.2 เฟส 2: ระบบสร้างแบบฝึกหัดเชิงโต้ตอบ (Interactive Exercise Generator) (ระยะเวลา: 6 สัปดาห์)

วัตถุประสงค์: ขยายขีดความสามารถของ AI Tutor ให้สามารถสร้างโจทย์การเขียนโปรแกรมเชิงวัตถุที่หลากหลายได้ตามหัวข้อที่ผู้ใช้เลือก พร้อมทั้งให้คำใบ้และประเมินคำตอบเบื้องต้นได้.

เพื่อให้แบบฝึกหัดที่สร้างขึ้นมีคุณค่าทางการศึกษาสูงสุด การพัฒนาในเฟสนี้จะไม่ใช่แค่การสร้างโจทย์แบบสุ่ม แต่จะเป็น "เครื่องมือสร้างโจทย์ที่มุ่งเป้าไปยังความเข้าใจผิดที่พบบ่อย" (Misconception-Targeted Problem Generator). จากการศึกษาพบว่านักศึกษามักมีปัญหาในจุดที่เฉพาะเจาะจง เช่น "ความสับสนระหว่างคลาสและอ็อบเจกต์" (D02.06) หรือ "การยึดติดกับวิธีคิดแบบโครงสร้าง" (D04.01). ดังนั้น แผนการพัฒนาจึงต้องรวมถึงการออกแบบ Prompt Template ที่สั่งให้ LLM สร้างสถานการณ์หรือโจทย์ที่บังคับให้นักศึกษาต้องเผชิญหน้าและแก้ไขความเข้าใจผิดเหล่านี้โดยตรง. ตัวอย่างเช่น สำหรับปัญหา D02.06, Prompt จะสั่งให้สร้างโจทย์ที่นักศึกษาต้องสร้างอ็อบเจกต์หลายๆ ตัวจากคลาสเดียว เพื่อพิสูจน์ว่าพวกเขาเข้าใจความแตกต่างระหว่าง "พิมพ์เขียว" (Class) และ "สิ่งที่สร้างจากพิมพ์เขียว" (Object).

แผนการดำเนินงานรายสัปดาห์:

- **สัปดาห์ที่ 1-2: Prompt Engineering & Design**
    - **กิจกรรม:** ออกแบบและพัฒนา Prompt Templates สำหรับการสร้างโจทย์ประเภทต่างๆ เช่น แบบฝึกหัดเติมโค้ดในช่องว่าง, โจทย์แก้ไขบั๊ก, และโจทย์เขียนคลาสทั้งหมดจากคำอธิบายสถานการณ์.
    - **กิจกรรม:** ประยุกต์ใช้เทคนิค Persona Prompting (เช่น "คุณคืออาจารย์สอน Python ผู้เชี่ยวชาญ...") และ Few-Shot Prompting (การให้ตัวอย่างโจทย์ที่ดี) เพื่อควบคุมคุณภาพและรูปแบบของโจทย์ที่ LLM สร้างขึ้น.
    - **กิจกรรม:** กำหนดให้ LLM ส่งผลลัพธ์เป็นโครงสร้างแบบ JSON ที่ชัดเจน เพื่อให้ Backend สามารถนำไปประมวลผลต่อได้ง่าย.
- **สัปดาห์ที่ 3-4: Backend Logic & State Management**
    - **กิจกรรม:** พัฒนาระบบ Backend เพื่อรับหัวข้อที่ผู้ใช้เลือก (เช่น "Inheritance", "Polymorphism") และเรียกใช้ Prompt Template ที่สอดคล้องกัน.
    - **กิจกรรม:** สร้างตรรกะสำหรับการสร้างคำใบ้ (Hint Generation) โดยใช้ Prompt อีกชุดหนึ่งที่แยกต่างหาก ซึ่งจะทำงานเมื่อผู้ใช้ร้องขอ.
    - **กิจกรรม:** (ทางเลือก) เริ่มออกแบบโครงสร้างข้อมูลสำหรับระบบติดตามความก้าวหน้าของผู้เรียน เพื่อบันทึกว่าผู้ใช้ทำแบบฝึกหัดหัวข้อใดไปแล้วบ้าง.
- **สัปดาห์ที่ 5-6: UI Integration & User Testing**
    - **กิจกรรม:** เพิ่มส่วนของแบบฝึกหัดใน UI ของ Streamlit ซึ่งประกอบด้วยพื้นที่แสดงโจทย์, พื้นที่สำหรับเขียนโค้ด, และปุ่มสำหรับ "ขอคำใบ้" และ "ส่งคำตอบ".
    - **กิจกรรม:** ผนวกรวม UI เข้ากับ Backend ที่พัฒนาขึ้น เพื่อให้ระบบทำงานได้อย่างสมบูรณ์.
    - **กิจกรรม:** จัดการทดสอบฟังก์ชันกับกลุ่มผู้ใช้ตัวอย่าง (นักศึกษา) เพื่อรวบรวมความคิดเห็นและนำมาปรับปรุง.

3.3 เฟส 3: เครื่องมือวิเคราะห์โค้ด (Code Analysis Tool) (ระยะเวลา: 8 สัปดาห์)

วัตถุประสงค์: พัฒนาฟีเจอร์ขั้นสูงที่สามารถให้คำติชมเชิงลึกเกี่ยวกับ "คุณภาพ" ของโค้ด OOP ที่นักศึกษาส่ง ไม่ใช่แค่การตรวจสอบว่าโค้ดทำงานถูกต้องตามโจทย์หรือไม่ แต่ต้องสามารถวิเคราะห์และให้คำแนะนำตามหลักการออกแบบซอฟต์แวร์ที่ดีได้.

เป้าหมายของเฟสนี้คือการสอนหลักการออกแบบ ซึ่งเป็นเรื่องที่ซับซ้อนและขึ้นอยู่กับบริบท ต่างจากข้อผิดพลาดทาง **синтаксис** ที่สามารถตัดสินถูกผิดได้ชัดเจน. การให้คำติชมที่มีคุณภาพเกี่ยวกับการออกแบบ เช่น "Object-Orientation

Abusers" หรือ "Couplers" ไม่สามารถทำได้ด้วย Prompt ง่ายๆ เช่น "ช่วยรีวิวโค้ดนี้". เพื่อให้ได้ผลลัพธ์ที่มีคุณค่าทางการศึกษาอย่างแท้จริง ระบบจะต้องจำลองกระบวนการคิดของนักพัฒนามืออาชีพเมื่อทำการ Code Review. ซึ่งทำได้โดยการใช้เทคนิคการทำ Prompt แบบ "Chain-of-Thought" (CoT) หรือแนวทางแบบโสคราตีส (Socratic Method). แทนที่จะใช้ Prompt เดียว, เราจะสร้าง "Code Review Prompt Chain" ซึ่งเป็น Prompt ที่มีโครงสร้างหลายขั้นตอน สั่งให้ LLM ทำการวิเคราะห์อย่างเป็นระบบ เช่น: ขั้นตอนที่ 1: ตรวจสอบหลักการ Encapsulation. ขั้นตอนที่ 2: ประเมินลำดับชั้นการสืบทอด. ขั้นตอนที่ 3: ระบุ Code Smells จากรายการที่กำหนดให้. ขั้นตอนที่ 4: สังเคราะห์คำแนะนำเพื่อการปรับปรุง. วิธีนี้จะบังคับให้ LLM แสดงเหตุผลอย่างเป็นขั้นเป็นตอน ทำให้คำติชมที่ได้มีความลึกซึ้ง, โปร่งใส, และเป็นประโยชน์ต่อการเรียนรู้.

แผนการดำเนินงานรายสัปดาห์:

- **สัปดาห์ที่ 1-3: Advanced Prompt R&D**
    - **กิจกรรม:** วิจัยและพัฒนา "Code Review Prompt Chain" โดยใช้เทคนิค CoT เป็นหลัก.
    - **กิจกรรม:** สร้าง Prompt ที่กำหนดให้ LLM (GPT-5-Codex) ตรวจสอบ Code Smells ที่พบบ่อยในงานของนักศึกษา เช่น Large Class, Feature Envy, Refused Bequest ตามที่ระบุไว้ในงานวิจัย.
    - **กิจกรรม:** ทดสอบและปรับปรุง Prompt อย่างต่อเนื่องโดยใช้ชุดข้อมูลตัวอย่างโค้ดที่หลากหลาย ทั้งโค้ดที่ดีและโค้ดที่มีปัญหาด้านการออกแบบ.
- **สัปดาห์ที่ 4-6: Backend Implementation & Result Parsing**
    - **กิจกรรม:** สร้าง API endpoint ในฝั่ง Backend ที่สามารถรับโค้ด Python จากผู้ใช้เป็น Input.
    - **กิจกรรม:** เขียนโค้ดเพื่อเรียกใช้ LLM API (GPT-5-Codex) ด้วย Prompt Chain ที่ออกแบบไว้.
    - **กิจกรรม:** พัฒนาระบบ Parser เพื่อแยกวิเคราะห์ผลลัพธ์ที่มีโครงสร้างซับซ้อนจาก LLM และจัดรูปแบบให้อยู่ในรูปแบบที่สามารถนำไปแสดงผลบน UI ได้อย่างสวยงามและเข้าใจง่าย.
- **สัปดาห์ที่ 7-8: UI for Feedback Visualization & Final Testing**
    - **กิจกรรม:** ออกแบบและสร้าง UI ใน Streamlit สำหรับแสดงผลการวิเคราะห์ โดยอาจแสดงโค้ดของผู้ใช้ควบคู่ไปกับคำติชมที่ได้รับแบบบรรทัดต่อบรรทัด.
    - **กิจกรรม:** พิจารณาใช้เทคนิคการไฮไลท์สีในส่วนของโค้ดที่เกี่ยวข้องกับคำแนะนำเพื่อเพิ่มความชัดเจน.
    - **กิจกรรม:** ทำการทดสอบระบบอย่างละเอียดกับกลุ่มเป้าหมาย ทั้งอาจารย์และนักศึกษา เพื่อประเมินคุณภาพและความถูกต้องของคำติชมที่ระบบสร้างขึ้น.

บทที่ 4: ตัวอย่างการออกแบบ Prompt (Prompt Engineering Examples)

4.1 Prompt สำหรับ RAG Q&A (เฟส 1)

- **System Prompt:**
- You are a helpful AI assistant for university students learning Object-Oriented Programming in Python. Use the provided context to answer the user's question accurately and concisely. If the answer is not in the context, say that you don't know.
- **Human Prompt:**
- Context:
- ---
- {retrieved_documents}
- ---
- Question: {user_question}
- Answer:
- **Rationale:** Prompt นี้ถูกออกแบบตามหลักปฏิบัติที่ดีที่สุด โดยมีการกำหนดบทบาท (Role) ของ AI อย่างชัดเจน, ตั้งข้อจำกัด (Constraint) ว่าให้ตอบจากข้อมูลที่ให้มาเท่านั้น, และใช้ตัวคั่น (Delimiter) --- เพื่อแยกส่วนของข้อมูลนำเข้าออกจากกันอย่างชัดเจน ทำให้ LLM สามารถเข้าใจและปฏิบัติตามคำสั่งได้ดียิ่งขึ้น.

4.2 Prompt สำหรับสร้างแบบฝึกหัด (เฟส 2)

- **Combined Prompt (Persona, Few-Shot, Structured Output):**

JSON

{

"role": "system",

"content": "You are an expert Python programming instructor creating exercises for a university-level Object-Oriented Programming course. Your goal is to create a challenging but fair coding problem that tests the student's understanding of a specific OOP concept. You must provide the output in a single JSON object."

},

{

"role": "user",

"content": "Generate a coding exercise about 'Hierarchical Inheritance'. Here are some examples of good exercises:\n\nExample 1 (Polymorphism):\n{\n \"topic\": \"Polymorphism\",\n \"title\": \"Polymorphic Zoo Animals\",\n \"problem_statement\": \"Create a base class `Animal` with a method `speak()`. Then, create two subclasses, `Dog` and `Cat`, that override the `speak()` method to return 'Woof!' and 'Meow!' respectively. Finally, write a function that takes a list of `Animal` objects and prints the

sound each animal makes.\",\n \"starter_code\": \"class Animal:\\n   def speak(self):\\n   pass\\n\\n# Your code here\",\n \"hints\":\n}\n\nNow, create a new exercise for the topic 'Hierarchical Inheritance'."

}

- **Rationale:** Prompt นี้ใช้เทคนิคขั้นสูงหลายอย่างร่วมกัน: 1) กำหนด **Persona** ที่ชัดเจน (ผู้สอนผู้เชี่ยวชาญ) เพื่อชี้นำโทนและคุณภาพของเนื้อหา. 2) ใช้เทคนิค **Few-Shot Prompting** โดยให้ตัวอย่างโจทย์ที่ดีเยี่ยมหนึ่งตัวอย่าง เพื่อเป็นแนวทางให้ LLM สร้างผลลัพธ์ที่มีโครงสร้างและคุณภาพในระดับเดียวกัน. 3) บังคับให้ผลลัพธ์เป็น **Structured Output** (JSON) ซึ่งทำให้ง่ายต่อการนำไปประมวลผลต่อในระบบ Backend. แนวคิดของ Hierarchical Inheritance มาจากหลักการพื้นฐานของ OOP.

4.3 Prompt สำหรับวิเคราะห์โค้ด (เฟส 3)

- **Chain-of-Thought (CoT) Prompt:**
- You are an expert code reviewer specializing in Object-Oriented Design Principles in Python. Analyze the following user-submitted code. Do not judge correctness, only design quality. Follow these steps and provide your analysis in a structured markdown format:
-
- **Step 1: Analyze Encapsulation.**
- - Are attributes properly hidden (e.g., using leading underscores)?
- - Is data accessed and modified through methods (getters/setters) rather than directly?
- - Identify any encapsulation violations.
-
- **Step 2: Analyze Inheritance.**
- - Is the class hierarchy logical (is-a relationship)?
- - Does the subclass properly call the parent's `__init__` method?
- - Identify any instances of "Refused Bequest" where a subclass ignores inherited methods it should use.
-
- **Step 3: Identify Common Code Smells.**
- - Based on the list, identify if any of these smells are present in the code. For each one you find, provide the name of the smell, the specific lines of code where it occurs, and a brief explanation of why it's a problem.
-
- **Step 4: Provide a Final Summary and Recommendations.**

- - Summarize the top 2-3 most critical design issues.
- - For each issue, provide a concrete suggestion for refactoring the code to improve its design.
-
- Here is the user's code:
- ```python
- {user_code}

Begin your analysis.

- **Rationale:** Prompt นี้เป็นตัวอย่างที่ยอดเยี่ยมของการใช้เทคนิค **Chain-of-Thought (CoT)** และการแบ่งงานที่ซับซ้อนออกเป็นขั้นตอนย่อยๆ. มันบังคับให้ LLM ปฏิบัติตามกระบวนการรีวิวโค้ดที่เป็นระบบเหมือนผู้เชี่ยวชาญ แทนที่จะให้ความเห็นแบบผิวเผิน. นอกจากนี้ยังมีการอ้างอิงถึง **Code Smells** ที่เฉพาะเจาะจงซึ่งระบุว่าเป็นปัญหาที่พบบ่อยสำหรับนักศึกษา ทำให้คำติชมที่ได้มีความเกี่ยวข้องและมีคุณค่าทางการศึกษาสูง.

บทที่ 5: ขั้นตอนถัดไป (Immediate Next Steps)

5.1 การตั้งค่าสภาพแวดล้อมโครงการ (Project Environment Setup Checklist)

- [ ] **Action:** สร้าง private repository ใหม่บน GitHub ชื่อ ai-oop-tutor.
- [ ] **Action:** Clone repository ลงมายังเครื่องพัฒนา local: git clone....
- [ ] **Action:** ตั้งค่า user name และ email สำหรับ Git ด้วยคำสั่ง git config.
- [ ] **Action:** สร้างโครงสร้างไดเรกทอรีของโปรเจกต์ตามแนวทางปฏิบัติที่ดีที่สุด :
- ai-oop-tutor/
- ├── .gitignore
- ├── README.md
- ├── requirements.txt
- ├── src/
- │ ├── __init__.py
- │ └── app.py
- └── tests/
- └── ...

- [ ] **Action:** สร้างและเปิดใช้งาน Python virtual environment: python -m venv.venv และ source.venv/bin/activate.
- [ ] **Action:** สร้างไฟล์ .gitignore โดยใช้ template มาตรฐานสำหรับ Python เพื่อไม่ให้ไฟล์ที่ไม่จำเป็น เช่น .venv, __pycache__, .env ถูกเพิ่มเข้าไปใน repository.
- [ ] **Action:** เพิ่มรายชื่อไลบรารีเริ่มต้นลงในไฟล์ requirements.txt: streamlit, python-dotenv, google-generativeai, langchain, faiss-cpu.
- [ ] **Action:** เขียนไฟล์ README.md ฉบับเริ่มต้นที่อธิบายวิสัยทัศน์ของโครงการและขั้นตอนการติดตั้ง.

5.2 การจัดหาทรัพยากร (Resource Acquisition)

- [ ] **Action:** สมัครและขอ API keys จาก Google AI Studio (สำหรับ Gemini) และ OpenAI Platform (สำหรับ GPT) และจัดเก็บคีย์เหล่านี้อย่างปลอดภัยในไฟล์ .env ซึ่งถูกระบุไว้ใน .gitignore แล้ว.
- [ ] **Action:** มอบหมายให้สมาชิกในทีมรับผิดชอบการรวบรวมคลังความรู้เริ่มต้นสำหรับเฟส 1. รวบรวมแหล่งข้อมูลคุณภาพสูงอย่างน้อย 3 แหล่ง (เช่น บทจากหนังสือ "Python Crash Course", เอกสารทางการของ Python เกี่ยวกับคลาส, เอกสารประกอบการบรรยายของมหาวิทยาลัย).

5.3 การวางแผนและกำหนดทีม (Planning and Team Assignment)

- [ ] **Action:** สร้างแผนผังเวลาของโครงการ (Project Timeline) โดยละเอียดโดยใช้ Gantt chart, ซึ่งแบ่งย่อยงานในแต่ละสัปดาห์ของทุกเฟส.
- [ ] **Action:** กำหนดบทบาทและความรับผิดชอบ:
    - **Project Lead:** ดูแลภาพรวมของไทม์ไลน์, งบประมาณ, และการสื่อสารกับผู้มีส่วนได้ส่วนเสีย.
    - **Lead Developer (AI/ML):** รับผิดชอบด้าน Prompt Engineering, การสร้าง RAG pipeline, และการเชื่อมต่อกับ LLM.
    - **Developer (Backend/UI):** รับผิดชอบการพัฒนา UI ด้วย Streamlit และตรรกะของ API ในฝั่ง Backend.
- [ ] **Action:** จัดการประชุมเปิดโครงการ (Project Kick-off Meeting) เพื่อทบทวนแผนงานนี้และยืนยันกำหนดการสำหรับ Sprint แรกของเฟส 1.